KERALA GOVERNMENT POLYTECHNIC COLLEGE KOZHIKODE-5

DEPARTMENT OF COMPUTER ENGINEERING

2019-2022

PROJECT REPORT ON

ROUTES

Submitted by

ADIDEV S R	19130432
ANJU O T K	19130445
ARJUN K V K	19130450
AROMAL K K	19130452
ASWIN RANJITH	19130454
AVINASH SANKAR S S	19130456
KIRANDEEP P	19130464

Guided by

NISHAMOL P H

KERALA GOVERNMENT POLYTECHNIC COLLEGE KOZHIKODE-5

Department of Computer Engineering

CERTIFICATE

This is to certify that Project entitled **ROUTES** *submitted by* **ADIDEV S R [Reg.No : 19130432], ANJU O T K [Reg.No : 19130445], ARJUN K V K [Reg.No : 19130450], AROMAL K K [Reg.No : 19130452], ASWIN RANJITH [Reg.No : 19130454],AVINASH SANKAR S S [Reg.No : 19130456],** *and* **KIRANDEEP P [Reg.No : 19130464]** *to the* **Department of Computer Science and Engineering, Kerala Government Polytechnic College Kozhikode-5** *, in partial fulfilment of the requirement for the award of Diploma under the Directorate of Technical Education,Government of kerala is a bonafide record of the work carried out by them.*

Mr. Shihabudheen K M

Head Of Department

Ms. Nishamol P H

Guide

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

It is with great enthusiasm and learning spirit that we are bringing out the report of our Project. Here we would like to mark our token of gratitude to all those who influenced us during the period of our work. We would like to express our sincere thanks to **Mr. Suresh Kumar J S**, Principal, Kerala Government Polytechnic College, Kozhikode for the facilities provided here.

With immense pleasure we express sincere thanks to **Mr. Shihabudheen K M**, Head, Department of Computer Engineering, for the entire motivation and support. We also extend our heart-felt gratitude to our guide **Ms. Nishamol P H**, Lecturer, Department of Computer Engineering, for the committed guidance, valuable suggestions. Also we extend our gratitude to all teachers in the Department of Computer Science and Engineering, Kerala Government Polytechnic College Kozhikode, for their support and inspiration.

And above all we praise and thank the Almighty God, who showered His abundant grace on us to make this work a success. We also express my special thanks and gratitude to our family and all our friends for their support and encouragement.

ABSTRACT

ROUTES is a web based application with a network of users using the service to discover new opportunities and sharing of ideas. This is a useful system for easily recruiting an employee. Nowadays many students pass out their graduate study, they never get an appropriate dream job. Similarly in a lot of companies they can't find sufficient skilled employees. The employees who have the right are not good enough.So it is intended to produce a mobile application and also a web portal to be called Job portal. It requires a web browser with an internet connection and a server supporting Java and a MySQL database.And also provides a mobile application for better user interface .It requires a Android or iOS operating system based mobile. ROUTES will have many features that will help users to interact between themselves and their colleagues. Some of the intended features which this platform will provide are,

1.Individuals and companies can communicate.

2.Registered companies can post Recruitment Program

3.Job seeker

TABLE OF CONTENTS

TITLE PAGE i

BONAFIDE CERTIFICATE i

ABSTRACT iii

LIST OF FIGURES vii

1 INTRODUCTION 1

1.1 FEATURES OF THIS SYSTEM 2

2 SYSTEM ANALYSIS 3

2.1 SYSTEM STUDY 3

2.1.1 EXISTING SYSTEM 3

2.1.2 PROPOSED SYSTEM 4

2.2 FEASIBILITY STUDY 4

2.2.1 OPERATIONAL FEASIBILITY 4

2.2.2 ECONOMIC FEASIBILITY 5

2.2.3 TECHNICAL FEASIBILITY 5

3 SYSTEM SPECIFICATION 6

3.1 INTRODUCTION 6

3.1.1 PURPOSE 6

3.1.2 SCOPE 6

3.1.3 DEFINITIONS,ACRONYMS, AND ABBREVIATIONS 7

3.1.4 OVERVIEW 7

3.2 OVERALL DESCRIPTION 7

3.2.1 PRODUCT PERSPECTIVE 7

3.2.2 PRODUCT FUNCTIONS 7
3.2.3 USER CHARACTERISTICS 8
3.2.4 PRINCIPLE ACTORS 8
3.2.5 GENERAL CONSTRAINTS 9
3.2.6 ASSUMPTION AND DEPENDENCIES 9
3.3 SPECIFIC REQUIREMENTS 9
3.3.1 FUNCTIONAL REQUIREMENTS 9
3.3.2 PERFORMANCE REQUIREMENTS 20
3.3.3 NON-FUNCTIONAL REQUIREMENTS 24
3.3.4 DESIGN CONSTRAINTS 26
3.3.5 FUTURE EXTENSIONS 26

4 SYSTEM DESIGN 27
4.1 MODULE DESCRIPTION 27
4.1.1 NORMAL USER 27
4.1.2 COMPANY USER 27
4.2 DATABASE DESIGN 28
4.3 DATA FLOW DIAGRAM 30
4.3.1 LEVEL 0 30
4.3.2 LEVEL 1 31
4.4 FORM DESIGN 32
4.4.1 Mobile UI 32
4.4.2 Web UI 33

5 SYSTEM IMPLEMENTATION AND RESULT 34
5.1 Important Codes 34
5.1.1 Log in(loginModel.dart) 34
5.1.2 Sign up (registrationModel.dart) 34
5.1.3 Add Job (jobDetailsModel.dart) 35
5.1.4 Post (viewPostModel.dart) 37
5.1.5 Apply job (applyjobsuccessModell.dart) 38
5.2 KMeans Clustering.java 40
5.3 RESULT 50

6 SYSTEM TESTING **51**

6.1 UNIT TESTING 51

6.2 INTEGRATION TESTING 52

6.3 SYSTEM TESTING 52

7 CONCLUSION **53**

8 FUTURE SCOPES **54**

9 REFERENCES **55**

LIST OF FIGURES

4.1 TABLES 28

4.2 LOGIN TABLE(USER AND COMPANY) 29

4.3 USER LOGIN ONLY 29

4.4 LOGIN TABLE(USER AND COMPANY) 29

4.5 USER LOGIN ONLY 30

4.6 LOGIN TABLE(USER AND COMPANY) 30

4.7 Level 0 30

4.8 Level 1 31

4.9 Mobile user interfaces 32

4.10 Web user interfaces 33

5.1 VIEW APPLICANT PAGE BEFORE SORTING 50

5.2 VIEW APPLICANT PAGE AFTER SORTING 50

CHAPTER 1

INTRODUCTION

As technology has developed, job searchers increasingly rely on internet job search platforms. We made the decision to create ”ROUTES” after considering the shortcomings of conventional systems and existing job search websites. In the suggested approach, we’re attempting to create a welcoming environment for both employees and employers that lessens the difficulties associated with locating a desired and acceptable employment in accordance with credentials and skill sets. By putting acceptable people in front of the organisation based on their qualifications and skill sets, we aim to lessen the hurdles. As a result, it will take less time for the companies to identify qualified candidates.In the proposed system, job seekers can upload their resumes in the required file format, see all the available jobs, search for desired jobs, and then apply for those jobs. On the other hand, this system enables employers to post their jobs and get a list of all applications which they can screen online, which reduces the huge amount of manual effort and time. Online recruitment, or e-recruitment, is turning out to be both job seekers’ and employers’ favourite activity, as offer and demand are well met in one place, and both must spend less time trying to get hold of the right roles or candidates. The company can post jobs, see applications and also sorted applicants list.

1.1 FEATURES OF THIS SYSTEM

Whether entering the job market for the first time or re-entering after a break or switching career, job search is a challenging task for employees.connecting with desirable candidates best fit for a job position is a challenging and important work for the employers.Our system develop an open community for employees and employers that reduces challenges in finding a desired job and filtering process that gives suitable candidates to the company based upon the qualifications and skill sets they mention.Some other features are:

1. To provide better job oppertunity for individuals.
2. Company can easily recruite individuals with desired skills and experience.
3. To provide better user interface and easy access.
4. To provide website access to companies.
5. To provide desired algorithm for sorting job applications.

CHAPTER 2

SYSTEM ANALYSIS

2.1 SYSTEM STUDY

2.1.1 EXISTING SYSTEM

The existing system enables jobseekers to search through print media like poster advertisements, newspapers and visual media like television or company websites for employment opportunities. This is a tedious task as it takes a lot of time and energy to search for the right job position, learn about the position and about the company. Job search for a proper match of skill set and salary is challenging. Job seekers can also find jobs through job fairs where they must first make it possible to attend the fairs which might be sometimes impossible with their schedules and if they visit the fairs they must hand over paper printed resumes. The more the number of candidates the more the number of papers for the company which is a lot of manual effort. Again, jobseekers might get job offers through placement cells in respective colleges but getting hold of the right opportunity at the right time is always challenging.

On the other hand, the same goes for employers who are looking for candidates who are best fitted for their job positions. They must constantly advertise, go to a lot of job fairs which still doesn't guarantee the best way to select from a large pool of candidates. Such conventional and outdated systems are replaced by severalwell featured national job search portals like Monster, Dice.com, Glassdoor, Indeed etc.All these job search and advertisement portals aims at e-recruitment by providing several simple and useful features to jobseekers and employers making job search and candidate selection a much time saving and easier process.

2.1.2 PROPOSED SYSTEM

With the advancement of technology, job seekers are relying greatly on online job search portals. Taking motivation from the conventional systems and their drawbacks and inspiration from the existing job search portals, we decided to develop "ROUTES". In the proposed system, we are trying to develop an open community for employees and employers that reduces challenges in finding a desired and suitable job according to qualifications and skill sets. We try to reduce the challenges by a filtering process that gives suitable candidates to the company based upon the qualifications and skill sets they mention. This in turn means less time-consuming as the companies find the deserving candidates. In the proposed system, job seekers can upload their resumes in the required file format, see all the available jobs, search for desired jobs, and then apply for those jobs. On the other hand, this system enables employers to post their jobs and get a list of all applications which they can screen online, which reduces the huge amount of manual effort and time. Online recruitment, or e-recruitment, is turning out to be both job seekers' and employers' favourite activity, as offer and demand are well met in one place, and both must spend less time trying to get hold of the right roles or candidates. The company can post jobs, see applications and check resumes in the proposed system.Proposed system has three modules Admin module,Company module,User module.

2.2 FEASIBILITY STUDY

Feasibility study is the test of system according to work ability impact on organization ability to meet user needs and effective use of resources

•Operational Feasibility

• Economic Feasibility

• Technical Feasibility

2.2.1 OPERATIONAL FEASIBILITY

Operation feasibility is used to check whether the project is operationally feasible or not. Our project is mainly different from the other system because of its web-support and sorting feature.So the measure for operational feasibility is something different from other system. Generally the operational feasibility is related to organization aspects.

2.2.2 ECONOMIC FEASIBILITY

Economical feasibility is the measure to determine the cost and benefit of the proposed system. A project is economical feasible which is under the estimated cost for its development. These benefits and costs may be tangible or intangible.Routes is the cost-effective project in which there is less possibility of intangible cost so there is no difficulty to determine the cost of the project.

2.2.3 TECHNICAL FEASIBILITY

Technical feasibility study is concerned with specifying equipment and software that will successfully satisfy the user requirement; the technical needs of the system may vary considerably.

All these technologies are easy to learn and can develop system very rapidly. After developing and deploying the system, any user can use the application.

CHAPTER 3

SYSTEM SPECIFICATION

3.1 INTRODUCTION

3.1.1 PURPOSE

Whether entering the job market for the first time or re-entering after a break or switching career, job search is a challenging task for employees.connecting with desirable candidates best fit for a job position is a challenging and important work for the employers.Our system develop an open community for employees and employers that reduces challenges in finding a desired job and filtering process that gives suitable candidates to the company based upon the qualifications and skill sets they mention.

3.1.2 SCOPE

SRS is intended to produce a mobile application and also a web portal to be called Job portal. It will have many features that will help users to interact between themselves and their colleagues. Some of the intended features which this platform will provide are.

1.Individuals and companies can communicate.

2.Registered companies can post Recruitment Program

3.Job seeker

3.1.3 DEFINITIONS,ACRONYMS, AND ABBREVIATIONS

Acronyms and Abbreviations

1 SRS: Software Requirements Specification

2 WWW: World Wide Weby

3.1.4 OVERVIEW

This SRS document has three sections **Section 1**(This section) provides an overview of the entire SRS document.

Section 2 gives a description of the general factors that affect the product that will be produced based on this SRS. It includes product perspective and functions, user characteristics, assumptions and dependencies of the product.

Section 3 addresses the specific and detailed requirements of the system to be produced. It includes the following

. • Functional Requirements which describe the inputs, process and outputs for each high level function.

• External interface requirements

• Other requirements

3.2 OVERALL DESCRIPTION

3.2.1 PRODUCT PERSPECTIVE

This product is a web based application with a network of users using the service to discover new opportunities and sharing of ideas. It requires a web browser with an internet connection and a server supporting Java and a MySQL database.And also provides a mobile application for better user interface .It requires a Android or iOS operating system based mobile.

3.2.2 PRODUCT FUNCTIONS

The application provides the following functions :

1. It allows the user to create his profile.

2. Users can communicate with each other.

3. It allows users to post blogs.

Class of use cases	Use cases	
Use case related to Installation	Installation	Creates and initializes working files
Use cases related to Login	Login	Login into ROUTES
Use case related to user	Use case related to user	Register new user
	View	View user details
	Edit user	Edit user details
User case related to Company	Register	Register company as user
	View	View company details
	Edit user	Edit company details
Use case related to Post	Add	User can add post
	View	User can view the post
	Edit	User can edit blogs
Use case related to follow	Add	User can follow other users
	View	User can view following users
	Edit Other users	User can unfollow
Use case related to Comment	Add	User can comment the post
	View	User can view comment
	Edit	User can edit posted comment
Use case related like	Edit	User can add like to post
User can add like to post	Send	User can share post
Use case related to Admin	View	Admin view user username
	Sort	Admin sort the resume using Specific algorithm
	Send	Admin send the sorted list to companies
	Authorisation of companies	Admin verify the companies

3.2.3 USER CHARACTERISTICS

This software is going to be used by students,learners,europatours and companies who browse the net and have Registered themselves at Routes. There are two types of users. They are Administrator and User.The various functions that can be performed by these users are indicated in section 3, functional requirements.

3.2.4 PRINCIPLE ACTORS

User Characteristics actors in AD are “User” , “Admin” and “Emergency unit”.

3.2.5 GENERAL CONSTRAINTS

1. It requires minimum internet connection.

2. Routes is a multi-user community system.

3.2.6 ASSUMPTION AND DEPENDENCIES

1. Only registered users can use the app.

3.3 SPECIFIC REQUIREMENTS

3.3.1 FUNCTIONAL REQUIREMENTS

Use case related to Installation

Use Case 1: Installation

Primary Actor: User

Pre Condition: Internet connection available.

Main Scenario:

1. User initiates App installation program.

2. System asks the user for the home directory in which all the working files will be created. Users are also asked for the initial login and password.

3. User specifies the home directory and login/password

Use cases related to system login:

Use Case 2: Login

Primary Actor: User

Pre Condition: Nil

Main Scenario:

1. Start the application. User prompted for login and password.

2. User gives the login and password.

3. System does authentication.

4. Main screen is displayed.

Alternate Scenario :

1. 4(a). Authorization fails :

Input	Processing	Output
Username and password of Admin,user,company.	Checks whether the Username and Password is valid .	Homepage .

a. 4(a)1. Prompt the user that he typed the wrong password

b. 4(a)2. Allow him to re-enter the password. Give him 3 chances.

Use case related to user

Use Case 3: Registration

Primary Actor: User

Pre Condition: Nil

Main Scenario:

Input	Processing	Output
Name,place,Address, phone number	Entered data is stored to the database	Details of the registered users

1. When user select registration option
2. System asks for firstname,lastname,place,post,pin,phone number,username and password in the first page.
3. In the next page the system asks for Educational and Professional details.
4. System save data into database
5. An alert message shoes “Successfully registered”

Alternate scenario:

6. Verification fails

7. Registration abort

Use Case 4: View User

Primary Actor:Admin

Pre Condition: nil

Main Scenario:

Input	Processing	Output
click on view user	Loads the database containing user details.	Details of registered users.

1. Admin selects the option of viewing view users

2. System displays name,address and phone number of the registered users

Alternate scenario:

1.Database crash

Use Case 5 : Edit user

Primary Actor:User

Pre Condition: Userlogged in

Main Scenario:

Input	Processing	Output
Name,Place, Phone number , E-mail.	Entered data replaces the user data .	Details of the edited user with status.

1. User select edit option System shows the user info with edit option
2. User can edit details and save
3. User details will be edited

Alternate scenario:

Use Case 6: Registration

Primary Actor:User-Company

Pre Condition: Nil

Main Scenario:

Input	Processing	Output
Company Name,Place,Other info	Entered data is stored to the database	Details of the edited user with status.

Alternate scenario:

Use Case 7: View Company

Primary Actor:Admin

Pre Condition: nil

Main Scenario:

Input	Processing	Output
click on view user-company	Loads the database containing user details.	Details of registered users.

1.Admin selects the option of viewing view users

2.System displays name,address and phone number of the registered users

Alternate scenario:

Use Case 8 : Edit Company

Primary Actor:User-Company

Pre Condition: Userlogged in

Main Scenario:

Input	Processing	Output
Name,Place, Phone number , E-mail.	Entered data replaces the user data .	Details of the edited user with status.

1.User select edit option System shows the user info with edit option

2.User can edit details and save

3.User details will be edited

Alternate scenario:

Use case related to Notification

Use Case 9:Send

Primary Actor:Admin

Pre Condition: Admin logged in

Main Scenario:

Input	Processing	Output
Notification to be sent	System adds new updates into database	Notification sent

Use Case 10:View

Primary Actor:User

Pre Condition:User logged in

Main Scenario:

Input	Processing	Output
System detects new update from database	System sent notification to the desired user	Desired user view the notification

1. User must select the notification option in the bottom menu
2. System display notifications

Use Case 11:Add

Primary Actor:User

Pre Condition:User logged in

Main Scenario:

Input	Processing	Output
User\upload,photos,videos,blogs	Entered data is stored into the database	Details of uploaded post as blogs

1. System adds the data into the database.

Alternate scenario:

Use Case 12:View

Primary Actor:User

Pre Condition:User logged in

Main Scenario:

Input	Processing	Output
Enter Home Page	Loads the page with blogs added by user	Page with blogs

Alternate scenario:

Use Case 13:Edit

Primary Actor:User

Pre Condition:User logged in

Main Scenario:

1. User can add or delete post.
2. The added post is viewed in the home page
3. User can delete added post from profile.

Input	Processing	Output
Update or Delete post	Data is deleted or updated from database	Updated page

Use case related to Follow

Use Case 14:Add

Primary Actor:User

Pre Condition:User logged in

Main Scenario:

1.User can follow other users.

Input	Processing	Output
User can follow other users	Update the current followers list in database	New following added

Use Case 15:View

Primary Actor:User

Pre Condition:

Main Scenario:

1. User can also unfollow others users which is already follows.

Input	Processing	Output
View profile	Update the followers list in database	Reload the page with updated following list

Alternate scenario:

Use case 16:Edit

Primary Actor:

Pre Condition:

Main Scenario:

Input	Processing	Output
User can unfollow other users	Update the followers list in database	Reload the page with updated following list

Alternate scenario:

Use case related to Comment

Use case 17:Add

Primary Actor:User

Pre Condition:

Main Scenario:

Input	Processing	Output
User can comment under every blogs	New comment is added to the database	Comment box updated with new comments

Alternate scenario:

Use case 18:View

Primary Actor:User

Pre Condition:user logged in

Main Scenario:

Input	Processing	Output
Add comment	Updating comment box with new comment	New comment is displayed on comment box

Alternate scenario:

Use case 19:edit

Primary Actor:User

Pre Condition:

Main Scenario:

Input	Processing	Output
User can delete their own comments	Currently deleted comment is deleted from database	Comment box reload with new updates

Use case related to share

Use case 20:Send

Primary Actor:User

Pre Condition:

Main Scenario:

Input	Processing	Output
User can share the blog to other social medias	Shared blog is updated with No.of share in database	A pop-window with which social media has to be shared.

Alternate scenario:

Use case related to Like post

Use case 21:Like

Primary Actor:User

Pre Condition:

Main Scenario:

Input	Processing	Output
Like post	Update the like count in database	Like button count incremented

Alternate scenario:

Use case 22:View

Primary Actor:Admin

Pre Condition:

Main Scenario:

Input	Processing	Output
Click on hyperlink to view feedback	Click on hyperlink to view feedback	Admin can view the feedbacks written by the users

Alternate scenario:

Use case related to

Admin **Use case 23:View**

Primary Actor:Admin

Pre Condition:

Main Scenario:

Input	Processing	Output
User resume	Store the resume in database	Update the resume list

Use case 24:Sort

Primary Actor:Admin

Pre Condition:

Main Scenario:

Use case 25:Send

Primary Actor:Admin

Input	Processing	Output
Resume that stored on the database	Sorting algorithm and store on database	Sorted list

Pre Condition:

Main Scenario:

Input	Processing	Output
Sorted resumes	Forward to companies page	resume by the sorted list

Use case 26:Authorisation of companies

Primary Actor:Admin

Pre Condition:

Main Scenario:

Input	Processing	Output
Company details	Verifying genuinity of the company	Company is verified

3.3.2 PERFORMANCE REQUIREMENTS

HARDWARE AND SOFTWARE REQUIREMENTS

HARDWARE REQUIREMENTS

• Processor : Intel(R) Core(TM) i3-3110M CPU@2.40GHZ(or above)

• Memory(RAM) : 4.00 GB

• Hard disk : 500 GB

• Input devices : Keyboard,touch pad

• Output devices :Monitor,Andriod Phone with basic functionality

SOFTWARE REQUIREMENTS

Server Side (Web Application)

• Frontend : JSP (using Java)

• Backend : MySQL

• Web Server : GlassFish

• OS : Windows, Mac OS, Linux etc.

Mobile Side

• Frontend : Flutter

• Language : Dart

• OS : Android/iOs

• IDE : Visual Studio Code

FORNT END

FLUTTER

Flutter, a simple and high performance framework based on Dart language, provides high performance by rendering the UI directly in the operating system's canvas rather than through native framework. The first version of Flutter was known by the codename "Sky" and ran on the Android operating system. It was unveiled at the 2015 Dart developer summit with the stated intent of being able to render 8 consistently at 120 frames per second. During the keynote of Google Developer Days in Shanghai in September 2018, Google announced Flutter Release Preview 2, which is the last big release before Flutter 1.0. On December 4th of that year, Flutter 1.0 was released at the Flutter Live event, denoting the first "stable" version of the Framework. On December 11, 2019, Flutter 1.12 was released at the Flutter Interactive event. On May 6, 2020, the Dart software development kit (SDK) in version 2.8 and the Flutter in version 1.17.0 were released, where support was added to the Metal API, improving performance on iOS devices (approximately 50

On March 3, 2021, Google released Flutter 2 during an online Flutter Engage event. This major update brought official support for web-based applications with new CanvasKit renderer and web specific widgets, early-access desktop application support for Windows, macOS, and Linux and improved Add-to-App APIs. This release included sound null-safety, which caused many breaking changes and issues with many external packages, but the Flutter team included instructions to mitigate these changes as well. On September 8th, 2021, the Dart SDK in version

2.14 and Flutter version 2.5 were released by Google. The update brought improvements to the Android Full-Screen mode and the latest version of Google's Material Design called Material You. Dart received two new updates, the newest lint conditions have been standardized and preset as the default conditions as well Dart for Apple Silicon is now stable.

ADVANTAGES

1. Large Community Support
2. Faster Code Writing
3. Own Rendering Engine
4. Cross Platform
5. Rich Libraries
6. Fast testing with hot reload

DART

Dart is a general-purpose, high-level modern programming language which is originally developed by Google. It is the new programming language which is emerged in 2011, but its stable version was released in June 2017. Dart is a dynamic, class-based, object-oriented programming language with closure and lexical scope. Syntactically, it is quite similar to Java, C, and JavaScript. Dart is an open-source programming language which is widely used to developthe mobile application, modern web-applications, desktop application, and the Internet of Things (IoT) using by Flutter framework.

JAVA

Java was originally developed by James Gosling at Sun Microsystems and released in May 1995 as a core component of Sun Microsystems' Java platform. The original and reference implementation Java compilers, virtual machines, and class libraries were originally released by Sun under proprietary licenses. Principles There were five primary goals in the creation of the Java language: 1. It must be simple, object-oriented, and familiar.

2. It must be robust and secure.
3. It must be architecture-neutral and portable.
4. It must execute with high performance.
5. It must be interpreted, threaded, and dynamic.

IDE

VISUAL STUDIO CODE

Visual Studio Code, also commonly referred to as VS Code, is a sourcecode editor made by Microsoft for Windows, Linux and macOS. Features include support for debugging, syntax highlighting, intelligent code completion, snippets, code refactoring, and embedded Git. Users can change the theme, keyboard shortcuts, preferences, and install extensions that add additional functionality. In the Stack Overflow 2021 Developer Survey, Visual Studio Code was ranked the most popular developer environment tool, with 7010 they use it. Visual Studio Code was first announced on April 29, 2015, by Microsoft at the 2015 Build conference. A preview build was released shortly thereafter. On November 18, 2015, the source of Visual Studio Code was released under the MIT License, and made available on GitHub. Extension support was also announced. On April 14, 2016, Visual Studio Code graduated from the public preview stage and was released to the Web. Microsoft has released most of Visual Studio Code's source code on GitHub under the permissive MIT License, while the releases by Microsoft are proprietary freeware.

NetBeans IDE

NetBeans IDE is a free and open source integrated development environment for application development on Windows, Mac, Linux, and Solaris operating systems.The IDE simplifies the development of web, enterprise, desktop, and mobile applications that use the Java and HTML5 platforms. The IDE also offers support for the development of PHP and C/C++ applications. NetBeans IDE offers first-class tools for Java web, enterprise, desktop, and mobile application development. It is consistently the first IDE to support the latest versions of the JDK, Java EE, and JavaFX. It provides smart overviews to help you understand and manage your applications, including ouf-of-the-box support for popular technologies such as Maven. With its end-to-end application development features, constantly improving Java Editor, and continual speed and performance enhancements, NetBeans IDE sets the standard for application development with cutting edge technologies out of the box.

BACK END

MySQL

MySQL is a relational database management system (RDBMS) developed by Oracle that is based on structured query language (SQL). A database is a structured collection of data. It may be anything from a simple shopping list to a picture gallery or a place to hold the vast amounts of information in a corporate network. In particular, a relational database is a digital store collecting data and organizing it according to the relational model. In this model, tables consist of rows and columns, and relationships between data elements all follow a strict logical structure. An RDBMS is simply the set of software tools used to actually implement, manage, and query such a database. MySQL is integral to many of the most popular software stacks for building and maintaining everything from customer-facing web applications to powerful, data-driven B2B services. Its open-source nature, stability, and rich feature set, paired with ongoing development and support from Oracle, have meant that internet-critical organizations such as Facebook, Flickr, Twitter, Wikipedia, and YouTube all employ MySQL backends. In a simple manner, SQL is a non-procedural, English-like language that processes data in groups of records rather than one record at a time.

WHY SQL USED IN THIS SYSTEM

SQL server can be used to develop both simple and complex database. Its wonderfully designed built-in wizard makes allow look like a product that any one can use. Although it is true that the simplest of the access application can be produced without any thought of design and without a single line of code written by the user, most applications require at least some designing and customer codes. It can store more data. So I used it in my project

3.3.3 NON-FUNCTIONAL REQUIREMENTS

Non-Functional requirements are the constraints or the requirements im-posed on the system. They specify the quality attribute of the software.

SECURITY

The system need to store the data in a secure location and store in a secure format. It requires a secure communication channel for the data, and the system needs to control the

system's sessions.

CONCURRENCY and CAPACITY

System should be able to recruit employees and helps the jobseekers to find dream job based on their skill and experience and potentially interacting with employees and employers.

PERFORMANCE

The performance of recruiting process of the system is depending on the accuracy in skill and experience of the jobseekers. We are using K-means algorithm which reaps an accuracy of 78.33 percentage

RELIABILITY

It is necessary to ensure and verify the systems reliability for our ROUTES system. We should use powerful RDBMS system, such as MySQL, to make the data secure from system crashes. This makes the system highly reliable.

MAINTAINABILITY

The system need to be regularly checked for preventive and corrective maintenance as it would come up with addition feature requirements or removing undesired events. Maintenance might specify scalability to grow and improve the system features and functionalities

USABILITY

End user satisfaction and acceptance is one of the key pillars that support a project success. As we are using Flutter framework, we can implement a simple yet powerful system and thereby it enhances the system usability

3.3.4 DESIGN CONSTRAINTS

RISK OF IDENTITY THEFT

There are loads of personal information that you have to display on your profile for prospective employers.

ATTRACTS BAD CANDIDATES

As previously mentioned, it's very easy for people to apply for jobs online; this can potentially be a negative point as well, though. Posting a position online usually results in hundreds of applicants, many of whom will not be suitable for or serious about the role, thus diluting the quality of your talent pool.

ATTRACTS FRAUDULENT APPLICANTS

Some applications might be fake to get information about the company or hiring officer (a common espionage tactic). Simultaneously, spammers might use the information posted online to promote a service or product to your company.

3.3.5 FUTURE EXTENSIONS

We can add more interactive session for user and company by providing live events and challanges. Using cloud storage system can improve reliability and performance. Features like suggestions, notifications, etc can be added. Two step verification of accounts can be added for enhanced privacy . Privacy can be reaped by privatization of user profile.

CHAPTER 4

SYSTEM DESIGN

4.1 MODULE DESCRIPTION

The complete project is divided into two modules. And the modularization is based on the type of the users in the system. The different modules based on the type of the users of the system are:

1.User module

2.Company module

4.1.1 NORMAL USER

1.Create a new account

2.Add ,view and delete post

3.Add,view and delete comments

4.Save Post and view saved post

5.Follow an account

6.Like and unlike a post

7.Apply for a job

4.1.2 COMPANY USER

1.Create a new account

2.Add ,view and delete post

3.Add,view and delete comments

4.Save Post and view saved post

5.Follow an account

6.Like and unlike a post

7.Add new job applications

4.2 DATABASE DESIGN

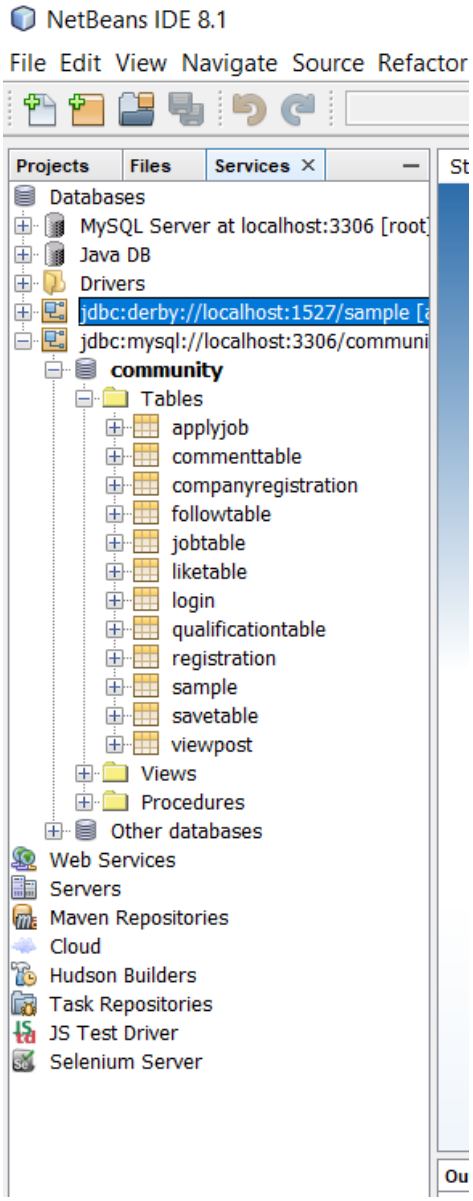
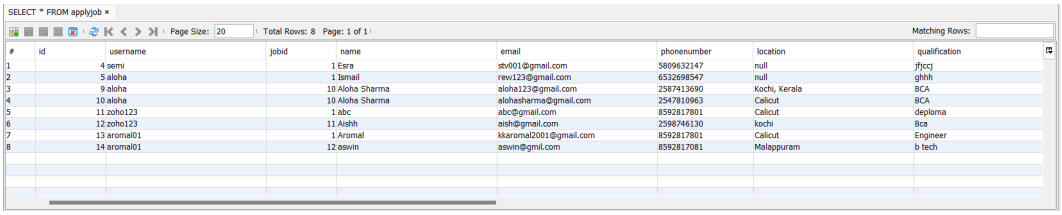


Figure 4.1: TABLES

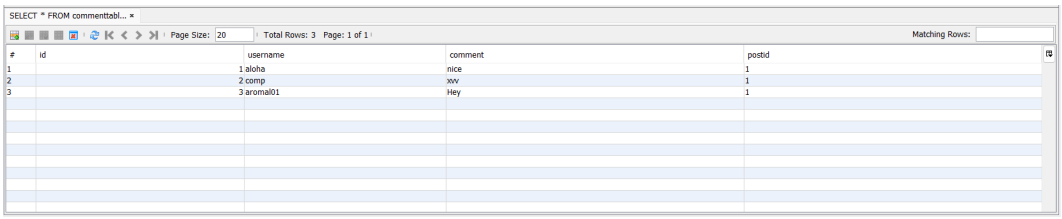


SELECT * FROM applyjob ×

Page Size: 20 Total Rows: 8 Page: 1 of 1 Matching Rows:

#	id	username	jobid	name	email	phonenumber	location	qualification
1	4	semi	1	Esra	stv001@gmail.com	5809632147	null	jfjccj
2	5	aloha	1	Ismail	rew123@gmail.com	6532698547	null	ghhh
3	9	aloha	10	Aloha Sharma	aloha123@gmail.com	2587413690	Kochi, Kerala	BCA
4	10	aloha	10	Aloha Sharma	alohasharma@gmail.com	2547810963	Calicut	BCA
5	11	zoho123	1	abc	abc@gmail.com	8592817801	Calicut	deploma
6	12	zoho123	11	Aishh	aish@gmail.com	2598746130	kochi	Bca
7	13	aromal01	1	Aromal	kkaromal2001@gmail.com	8592817801	Calicut	Engineer
8	14	aromal01	12	aswin	aswin@gmil.com	8592817081	Malappuram	b tech

Figure 4.2: LOGIN TABLE(USER AND COMPANY)

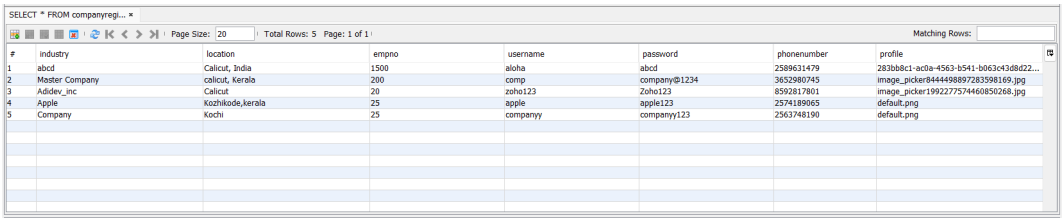


SELECT * FROM commenttabl... ×

Page Size: 20 Total Rows: 3 Page: 1 of 1 Matching Rows:

#	id	username	comment	postid
1		1 aloha	nice	1
2		2 comp	xvv	1
3		3 aromal01	Hey	1

Figure 4.3: USER LOGIN ONLY



SELECT * FROM companyregi... ×

Page Size: 20 Total Rows: 5 Page: 1 of 1 Matching Rows:

#	industry	location	empno	username	password	phonenumber	profile
1	abcd	Calicut, India	1500	aloha	abcd	2589631479	283bb8c1-ac0a-4563-b541-b063c43d8d22...
2	Master Company	calicut, Kerala	200	comp	company@1234	3652980745	image_picker8444498897283598169.jpg
3	Adidev_inc	Calicut	20	zoho123	Zoho123	8592817801	image_picker1992277574460850268.jpg
4	Apple	Kozhikode,kerala	25	apple	apple123	2574189065	default.png
5	Company	Kochi	25	companyy	companyy123	2563748190	default.png

Figure 4.4: LOGIN TABLE(USER AND COMPANY)

SELECT * FROM followtable ×

Page Size: 20 Total Rows: 11 Page: 1 of 1 Matching Rows:

#	id	profileuser	followeduser	type
1	2	sadique	sameena	1
2	3	abcd	sameena	1
3	4	efghi	sameena	1
4	25	sameena	aloha	1
5	35	rajj	comp	1
6	36	aloha	comp	1
7	38	semi	aloha	1
8	39	rajj	aloha	1
9	40	aloha	aloha	1
10	42	comp	aloha	1
11	45	zoho123	aromal01	1

Figure 4.5: USER LOGIN ONLY

SELECT * FROM jobtable ×

Page Size: 20 Total Rows: 9 Page: 1 of 1 Matching Rows:

#	id	position	location	qualification	responsibility	username	skills
1	1	Flutter Developer	Singapore, Permanent	BS or MS in Computer Science or related...	Work effectively as a member of self-org...	aloha	Flutter,Dart,Visual Studio Code
2	2	Android Developer	Singapore, Contract	BS or MS in Computer Science or related...	Work effectively as a member of self-org...	aloha	Android,Java,Kotlin,Android Studio
3	4	Python Developer	Singapore	jdofjckclx	Work effectively as a member of self-org...	aloha	Python,Django,Flask
4	5	PHP Developer	Calicut, India	BS or MS in Computer Science or related...	Work effectively as a member of self-org...	aloha	PHP,Netbeans,MySQL
5	9	null	null	null	null	null	null
6	10	Flutter Developer	Kochi, Kerala	Bca	skdjjkddk	comp	Flutter, Dart
7	11	Android Developer	Calicut	ddddddddddd	Aaaaaaaaaaaaa	comp	Android, Java
8	12	Flutter	kochi	fjfjfjc	assddf	companyy	flutter,dart
9	13	debugger	Calicut	flutter, Android studio	debugging codes	zoho123	Dart, Android,java

Figure 4.6: LOGIN TABLE(USER AND COMPANY)

4.3 DATA FLOW DIAGRAM

4.3.1 LEVEL 0

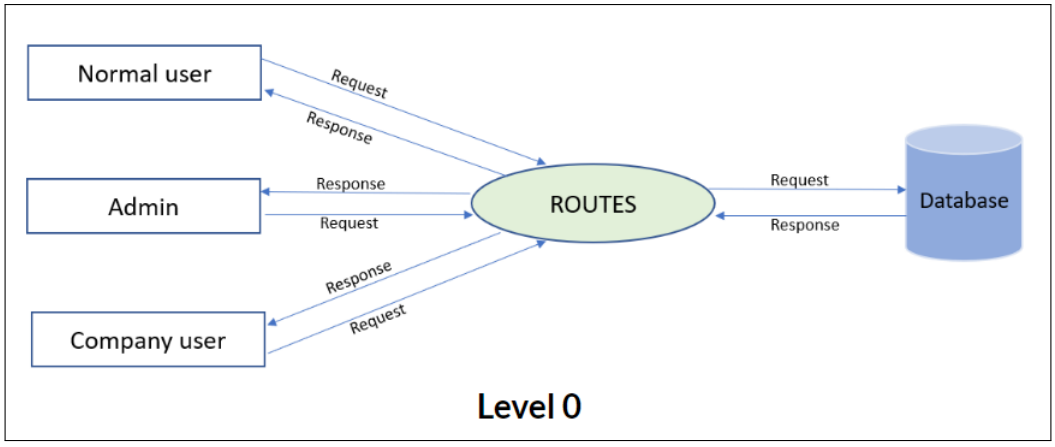


Figure 4.7: Level 0

4.3.2 LEVEL 1

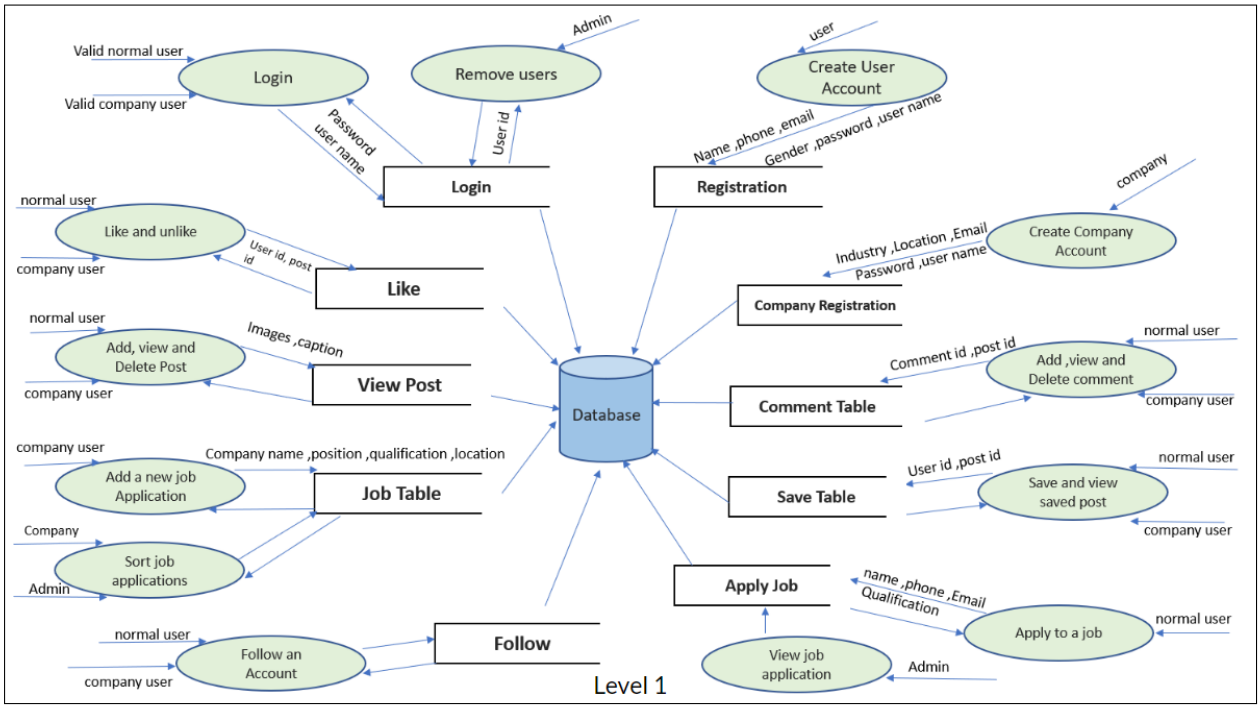


Figure 4.8: Level 1

4.4 FORM DESIGN

4.4.1 Mobile UI

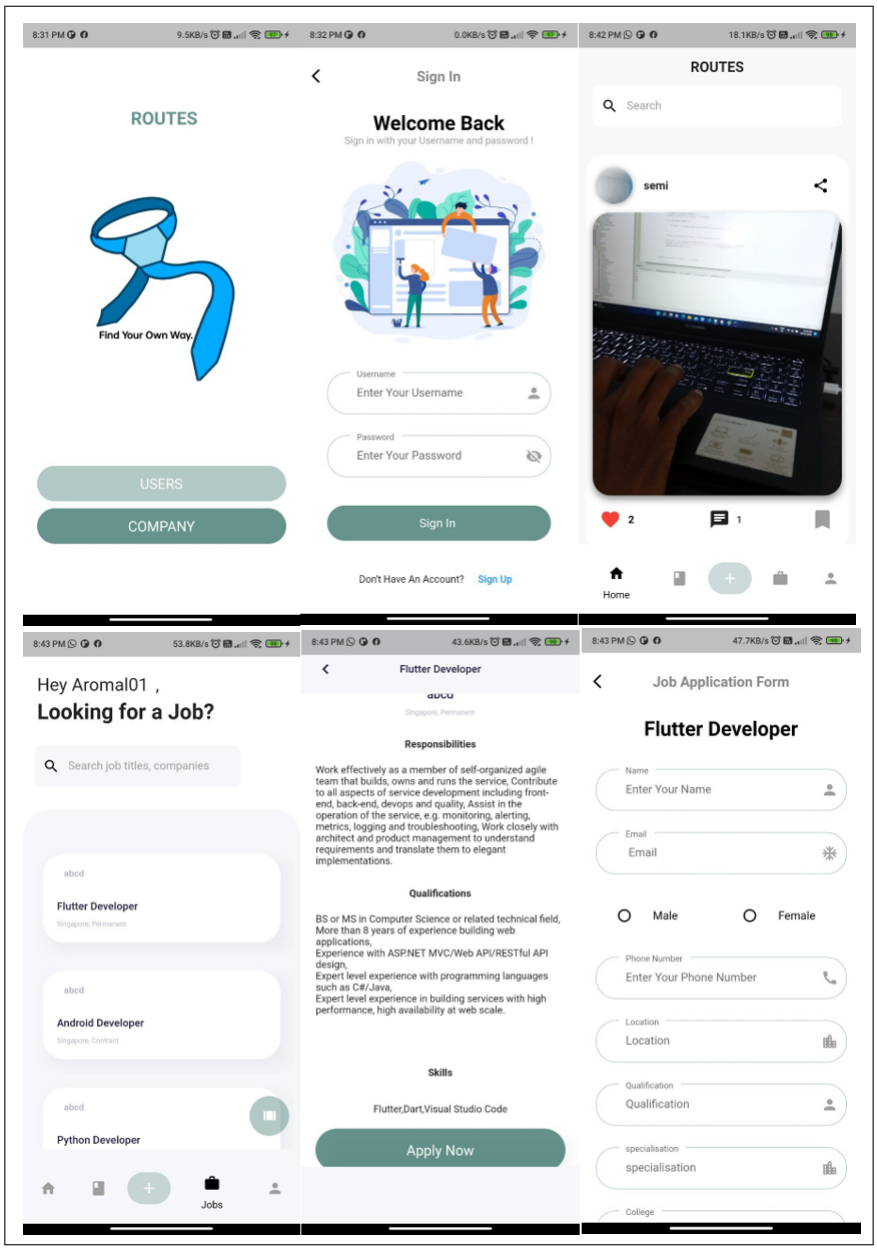


Figure 4.9: Mobile user interfaces

4.4.2 Web UI

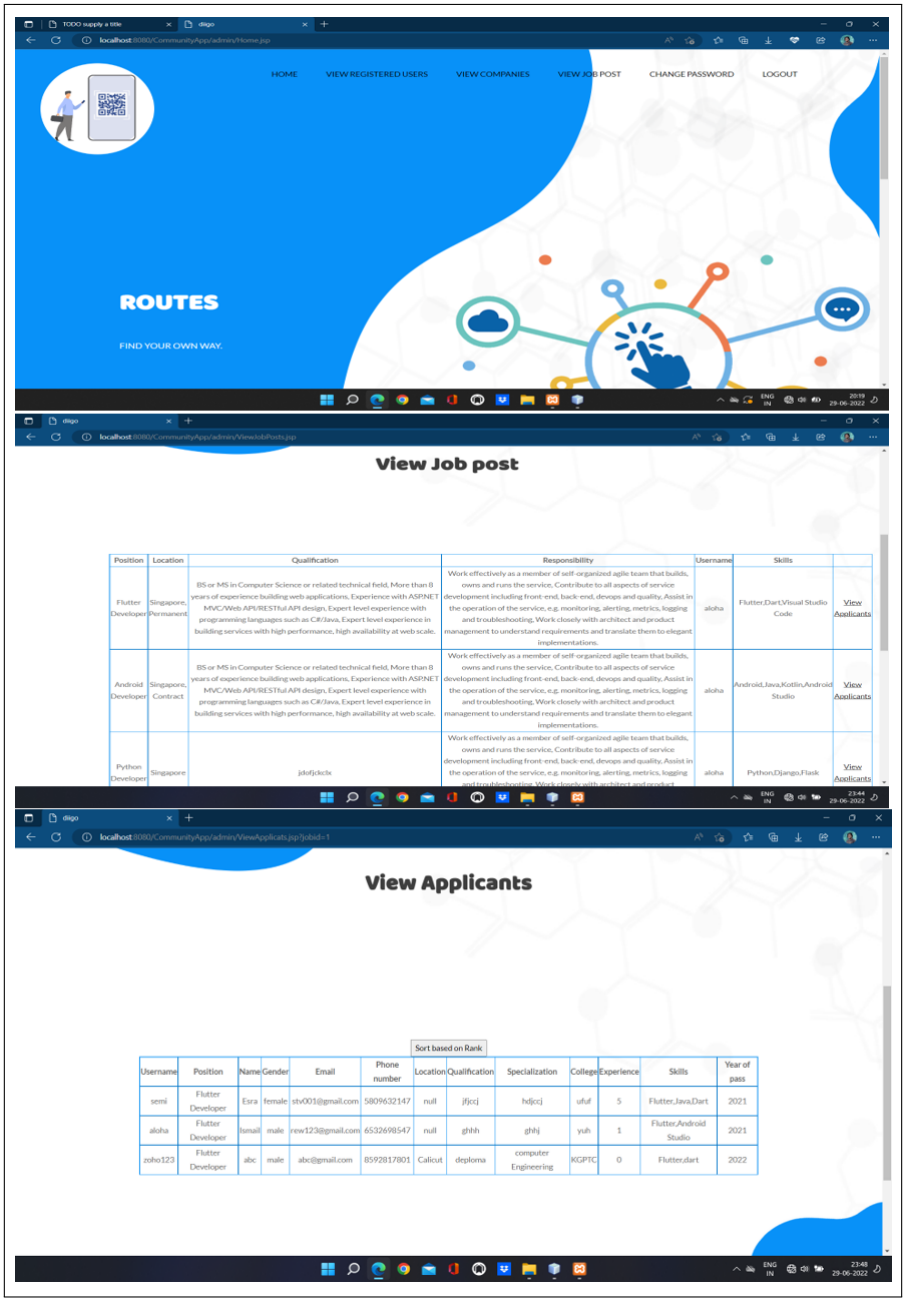


Figure 4.10: Web user interfaces

CHAPTER 5

SYSTEM IMPLEMENTATION AND RESULT

5.1 Important Codes

5.1.1 Log in(loginModel.dart)

```
class loginModel {
  final String username;
  final String password;
  final String type;
  loginModel({this.username, this.password, this.type});
  factory loginModel.fromJson(Map<String, dynamic> Json) {
    return loginModel(
        username: Json['username'],
        password: Json['password'],
        type: Json['type']);
  }
}
```

5.1.2 Sign up (registrationModel.dart)

```
class regModel {
  final String id;
  final String name;
  final String rollnoOrEmpno;
  final String college;
  final String phonenumber;
  final String username;
  final String password;
```

```
  final String gender;
  regModel(
      {this.id,
      this.name,
      this.rollnoOrEmpno,
      this.college,
      this.phonenumber,
      this.username,
      this.password,
      this.gender});
  factory regModel.fromJson(Map<String, dynamic> Json) {
    return regModel(
        id: Json['id'],
        name: Json['name'],
        rollnoOrEmpno: Json['rollnoOrEmpno'],
        college: Json['college'],
        phonenumber: Json['phonenumber'],
        username: Json['username'],
        password: Json['password'],
        gender: Json['gender']);
  }
}
```

5.1.3 Add Job (jobDetailsModel.dart)

```
class JobModel {
  final String id;
  // final String companylogo;
  final String industry;
  final String position;
  final String location;
  final String responsibility;
  final String qualification;
```

```
final String skills;
JobModel({
  this.id,
  // this.companylogo,
  this.industry,
  this.position,
  this.location,
  this.qualification,
  this.responsibility,
  this.skills
});
factory JobModel.fromJson(Map<String, dynamic> Json) {
  print((Json['id']));
  print((Json['industry']));
  print((Json['position']));
  print((Json['location']));
  print((Json['responsibility']));
  print((Json['qualification']));
  print((Json['skills']));
  try {
    return JobModel(
      id: Json['id'],
      // companylogo: Json['companylogo'],
      industry: Json['industry'],
      position: Json['position'],
      location: Json['location'],
      responsibility: Json['responsibility'],
      qualification: Json['qualification'],
      skills: Json['skills']
    );
  } catch (error) {
```

```
      print(error);
    }
  }
}
```

5.1.4 Post (viewPostModel.dart)

```
class ViewpostModel {
  final String id;
  final String username;
  final String userimage;
  final String image;
  final String post_title;
  final String post_contenct;
  final String date;
  final String likecount;
  final String profile;
  ViewpostModel(
      {this.id,
      this.username,
      this.userimage,
      this.image,
      this.post_title,
      this.post_contenct,
      this.date,
      this.likecount,
      this.profile
      });
  factory ViewpostModel.fromJson(Map<String, dynamic> Json) {
    print((Json['id']));
    print((Json['username']));
    print((Json['userimage']));
    print((Json['image']));
```

```
    print((Json['post_title']));
    print((Json['post_contenct']));
    print((Json['date']));
    print((Json['likecount']));
    print((Json['profile']));
    try {
      return ViewpostModel(
        id: Json['id'],
        username: Json['username'],
        userimage: Json['userimage'],
        image: Json['image'],
        post_title: Json['post_title'],
        post_contenct: Json['post_contenct'],
        date: Json['date'],
        likecount: Json['likecount'],
        profile: Json['profile']
      );
    } catch (error) {
      print(error);
    }
  }
}
```

5.1.5 Apply job (applyjobsuccessModell.dart)

```
class JobSuccessModel {
  final String username;
  final int jobid;
  final String position;
  final String name;
  final String gender;
  final String email;
  final String location;
```

```
final String phonenumber;
final String qualification;
final String specialisation;
final String college;
final String experience;
final String skills;
final String yearofpass;
JobSuccessModel(
    {this.username,
    this.position,
    this.jobid,
    this.name,
    this.gender,
    this.email,
    this.college,
    this.experience,
    this.phonenumber,
    this.location,
    this.qualification,
    this.skills,
    this.specialisation,
    this.yearofpass});
factory JobSuccessModel.fromJson(Map<String, dynamic> Json) {
  return JobSuccessModel(
      username: Json['username'].toString(),
      jobid: Json['jobid'],
      position: Json['position'].toString(),
      name: Json['name'].toString(),
      gender: Json['gender'].toString(),
      email: Json['email'].toString(),
      phonenumber: Json['phonenumber'].toString(),
```

```
        location: Json['location'].toString(),
        qualification: Json['qualification'].toString(),
        specialisation: Json['specialisation'].toString(),
        college: Json['college'].toString(),
        experience: Json['experience'].toString(),
        skills: Json['skills'].toString(),
        yearofpass: Json['yearofpass'].toString());
  }
}
```

5.2 KMeans Clustering.java

```
package com.packages;
import java.util.ArrayList;
import java.util.Vector;
public class KMeans_Clustering {
    public static final int NUM_CLUSTERS = 5;    // Total clusters.
    public static int TOTAL_DATA = 5;        // Total data points.
    private static double SAMPLES[][] = new double[][]{{1.0, 3.8},
    {4.2, 5.0},
    {5.6, 6.0},
    {6.8, 7.0},
    {7.2, 9.0},};
    Vector v = null;
    String type = "", sql = "";
    Vector studentsinCluster = new Vector();
    DBConnection db = null;
    public KMeans_Clustering(Vector dataset, String type)
{//argument constructor
        v = dataset;
        TOTAL_DATA = v.size();
        this.type = type;
        db = new DBConnection();
```

```
    }
    public ArrayList<Data> dataSet = new ArrayList<Data>();
    private ArrayList<Centroid> centroids =
new ArrayList<Centroid>();
    private void initialize() {
        System.out.println("Centroids initialized at:");
 centroids.add(new Centroid(70.1, 100.0)); // Excellent.
 centroids.add(new Centroid(60.51, 70.0)); // Good.
 centroids.add(new Centroid(50.51, 60.50)); // Average.
centroids.add(new Centroid(40.51, 50.50)); // Below Average.
centroids.add(new Centroid(1, 40.5)); // Need Improvement.
System.out.println("     (" + centroids.get(0).X() + ",
" + centroids.get(0).Y() + ")");
System.out.println("     (" + centroids.get(1).X() + ",
 " + centroids.get(1).Y() + ")");
System.out.print("\n");
return;
    }
    private void kMeanCluster() {
        final double bigNumber = Math.pow(10, 10);
        double minimum = bigNumber;
        double distance = 0.0;
        int sampleNumber = 0;
        int cluster = 0;
        boolean isStillMoving = true;
        Data newData = null;
        StScoreBean bean = null;
        while (dataSet.size() < TOTAL_DATA) Z
            bean = (StScoreBean) v.get(sampleNumber);
            double score = 0;
            double minScore = 0;
```

```
        System.out.println("score ::" + score);
        if (type.equals("overall")) {
            score = bean.getTotalscore();
            minScore = bean.getScore_min_range();
            sql="UPDATE applyjob set totalscore=
"+bean.getTotalscore()+" , review='"+bean.getRank()+
"' where username='"+bean.username+"' and jobid=
"+bean.jobid+"";
            System.out.println(sql);
            db.executeSQL(sql);
        }
        System.out.println("score after :: " + score);
        newData = new Data(minScore, score, bean);
        dataSet.add(newData);
        minimum = bigNumber;
        for (int i = 0; i < NUM_CLUSTERS; i++) {
            distance = dist(newData, centroids.get(i));
            if (distance < minimum) {
                minimum = distance;
                cluster = i;
            }
        }
        newData.cluster(cluster);
        // calculate new centroids.
        for (int i = 0; i < NUM_CLUSTERS; i++) {
            int totalX = 0;
            int totalY = 0;
            int totalInCluster = 0;
            for (int j = 0; j < dataSet.size(); j++) {
                if (dataSet.get(j).cluster() == i) {
                    totalX += dataSet.get(j).X();
```

```
                totalY += dataSet.get(j).Y();
                totalInCluster++;
            }
        }
        if (totalInCluster > 0) {
            centroids.get(i).X(totalX / totalInCluster);
            centroids.get(i).Y(totalY / totalInCluster);
        }
    }
    sampleNumber++;
}
// Now, keep shifting centroids until equilibrium occurs.
while (isStillMoving) {
    // calculate new centroids.
    for (int i = 0; i < NUM_CLUSTERS; i++) {
        int totalX = 0;
        int totalY = 0;
        int totalInCluster = 0;
        for (int j = 0; j < dataSet.size(); j++) {
            if (dataSet.get(j).cluster() == i) {
                totalX += dataSet.get(j).X();
                totalY += dataSet.get(j).Y();
                totalInCluster++;
            }
        }
        if (totalInCluster > 0) {
            centroids.get(i).X(totalX / totalInCluster);
            centroids.get(i).Y(totalY / totalInCluster);
        }
    }
    // Assign all data to the new centroids
```

```
            isStillMoving = false;

            for (int i = 0; i < dataSet.size(); i++) {
                Data tempData = dataSet.get(i);
                minimum = bigNumber;
                for (int j = 0; j < NUM_CLUSTERS; j++) {
                    distance = dist(tempData, centroids.get(j));
                    if (distance < minimum) {
                        minimum = distance;
                        cluster = j;
                    }
                }
                tempData.cluster(cluster);
                if (tempData.cluster() != cluster) {
                    tempData.cluster(cluster);
                    isStillMoving = true;
                }
            }
        }
        return;
    }
    /**
     * // Calculate Euclidean distance.
     *
     * @param d - Data object.
     * @param c - Centroid object.
     * @return - double value.
     */
    private static double dist(Data d, Centroid c) {
        return Math.sqrt(Math.pow((c.Y() - d.Y()), 2)
+ Math.pow((c.X() - d.X()), 2));
```

```
}
public static class Data {
    public double mX = 0;
    public double mY = 0;
    public int mCluster = 0;
    private StScoreBean jobseekerbean;
    /**
     * Get the value of studentbean
     *
     * @return the value of studentbean
     */
    public StScoreBean getStudentbean() {
        return jobseekerbean;
    }
    /**
     * Set the value of studentbean
     *
     * @param studentbean new value of studentbean
     */
    public void setStudentbean(StScoreBean studentbean) {
        this.jobseekerbean = studentbean;
    }
    public Data() {
        return;
    }
    public Data(double x, double y, StScoreBean bean) {
        this.X(x);
        this.Y(y);
        this.jobseekerbean = bean;
        return;
    }
```

```
        public void X(double x) {
            this.mX = x;
            return;
        }
        public double X() {
            return this.mX;
        }
        public void Y(double y) {
            this.mY = y;
            return;
        }
        public double Y() {
            return this.mY;
        }
        public void cluster(int clusterNumber) {
            this.mCluster = clusterNumber;
            return;
        }
        public int cluster() {
            return this.mCluster;
        }
    }
    private static class Centroid {
        private double mX = 0.0;
        private double mY = 0.0;
        public Centroid() {
            return;
        }
        public Centroid(double newX, double newY) {
            this.mX = newX;
            this.mY = newY;
```

```
            return;
        }
        public void X(double newX) {
            this.mX = newX;
            return;
        }
        public double X() {
            return this.mX;
        }
        public void Y(double newY) {
            this.mY = newY;
            return;
        }
        public double Y() {
            return this.mY;
        }
    }
    public void doClustering() {
        initialize();
        kMeanCluster();
        int[] cluster_count = new int[5];
        // Print out clustering results.
        for (int i = 0; i < NUM_CLUSTERS; i++) {
            System.out.println("Cluster " + i + " includes:");
            for (int j = 0; j < TOTAL_DATA; j++) {
                if (dataSet.get(j).cluster() == i) {
System.out.println("      (" + dataSet.get(j).X()
        + ", " + dataSet.get(j).Y() + ")");
                    cluster_count[i] = cluster_count[i] + 1;
                }
            }
```

```
            System.out.println();
        }
System.out.println("type ::" + type + " cluster_count[1] ::
" + cluster_count[1]);
        if (type.equals("overall")) {
System.out.println("cluster_count[0] =" + cluster_count[0]);
System.out.println("cluster_count[1] =" + cluster_count[1]);
System.out.println("cluster_count[2] =" + cluster_count[2]);
System.out.println("cluster_count[3] =" + cluster_count[3]);
System.out.println("cluster_count[4] =" + cluster_count[4]);
 /* frmPerformanceFinder.lblOverallExcellent.
setText("" + cluster_count[0]);
frmPerformanceFinder.lblOverallGood.
setText("" + cluster_count[1]);
frmPerformanceFinder.lblOverallAverage.
setText("" + cluster_count[2]);
frmPerformanceFinder.lblOverallBelowAverage.
setText("" + cluster_count[3]);
frmPerformanceFinder.lblOverallNeedImprovement.
setText("" + cluster_count[4]);
             */
        }
        // Print out centroid results.
        System.out.println("Centroids finalized at:");
        for (int i = 0; i < NUM_CLUSTERS; i++) {
            System.out.println("      (" + centroids.get(i).X()
 + ", " + centroids.get(i).Y());
        }
        System.out.print("\n");
        return;
    }
```

```
    public ArrayList<KMeans_Clustering.Data> getDataSet() {
        return dataSet;
    }
    public static void main(String[] args) {
        /*initialize();
         kMeanCluster();
         // Print out clustering results.
         for (int i = 0; i < NUM_CLUSTERS; i++) {
         System.out.println("Cluster " + i + " includes:");
         for (int j = 0; j < TOTAL_DATA; j++) {
         if (dataSet.get(j).cluster() == i) {
         System.out.println("      (" + dataSet.get(j).X() + ",
" + dataSet.get(j).Y() + ")");
         }
         } // j
         System.out.println();
         } //
         // Print out centroid results.
         System.out.println("Centroids finalized at:");
         for (int i = 0; i < NUM_CLUSTERS; i++) {
         System.out.println("      (" + centroids.get(i).X()
 + ", " + centroids.get(i).Y());
         }
         System.out.print("\n");
         return;
         */
    }
}
```

5.3 RESULT

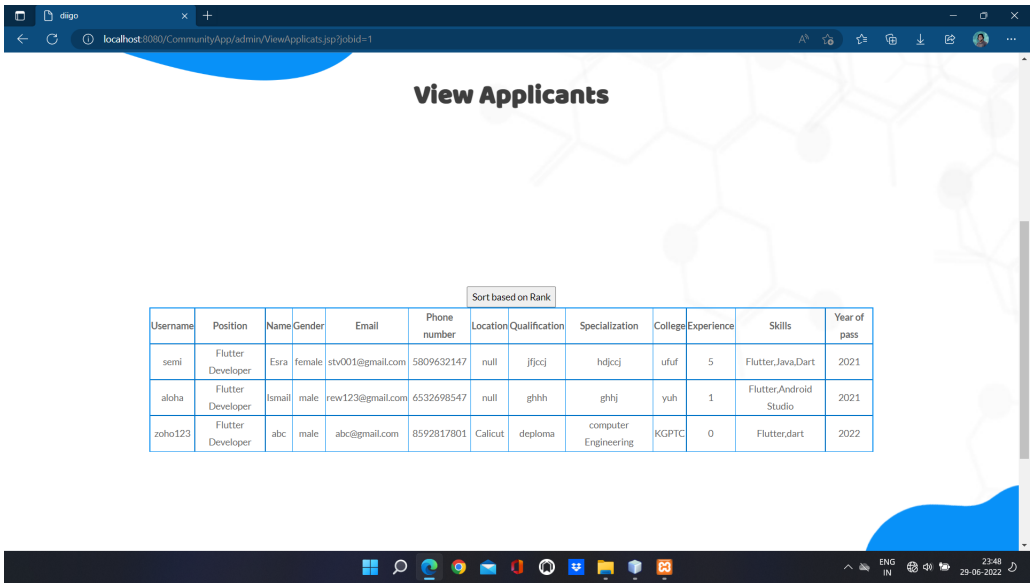


Figure 5.1: VIEW APPLICANT PAGE BEFORE SORTING

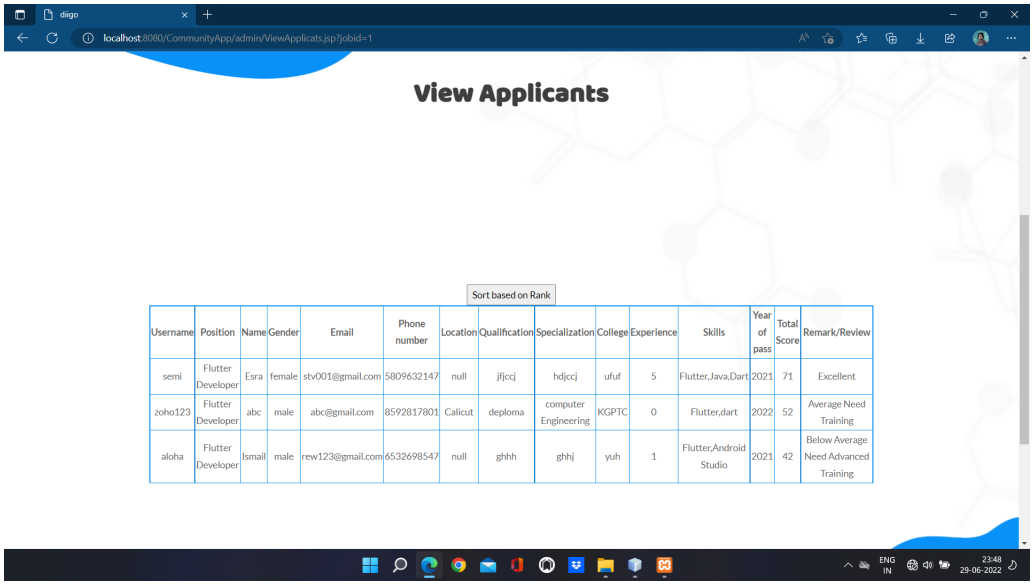


Figure 5.2: VIEW APPLICANT PAGE AFTER SORTING

CHAPTER 6

SYSTEM TESTING

6.1 UNIT TESTING

SI NO	TEST CASE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	REMARKS
1	Test Case for adding valid login credentials	Valid username & password	Redirected to Home Page.	Redirected to Home Page.	PASS
2	Test Case for adding invalid login credentials	invalid username & password	Do not redirected to Home Page.	Do not redirected to Home Page.	PASS
3	Test case for User.	Valid name, college,phone number,username, password, gender	Redirects to Login Page	Redirects to Login Page	PASS
4	Test case for Company User.	Valid Company name, location, username, password,phone number	Redirects to Login Page.	Redirects to Login Page.	PASS

SI NO	TEST CASE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	REMARKS
5	Test case for adding profile photo	Adding photo from gallery or camera	Profile picture added	Profile picture added	PASS
6	Test case for adding post	Adding photo from gallery or camera	New post is added.	New post is added.	PASS
7	Test case for delete post	None	Post Deleted	Post Deleted	PASS
8	Test case for adding comment	Adding comment for post.	New comment added.	New comment added.	PASS

6.2 INTEGRATION TESTING

SI NO	TEST CASE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	REMARKS
1	Test case for apply jobs.	Name,email,phone number,qualification,specialisation,college,skills,year of pass.	View the job on applied job.	View the job on applied job.	PASS
2	Test case for add jod.	Position,location,qualification,responsibility,skills	View the job on job views.	View the job on job views.	PASS
3	Test case for searching user post.	Valid username	Display user post.	Display user post.	PASS

6.3 SYSTEM TESTING

SI NO	TEST CASE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	REMARKS
1	Load app with internet connection	None	App load perfectly	App load perfectly	PASS
2	Load app without internet connection	None	App didn't load	App didn't load	PASS
3	Sorting applied users based on criteria mentioned in algorithm	Applied users	Sorted according to preferred criteria	Sorted according to preferred criteria	PASS

.

CHAPTER 7

CONCLUSION

ROUTES stands as a revolutionizing element in the sphere of recruitment. They act as a communication bridge between applicants and recruiters facilitating their requirements. This application helps organizations to have a greater exposure to the candidate pool and also job seekers facilitating wide search of jobs matching their interests. The android application provides flexibility to the jobseekers to view the openings and applied jobs.Our system develop an open community for employees and employers that reduces challenges in finding a desired job and filtering process that gives suitable candidates to the company based upon the qualifications and skill sets they mention. This application provides an enhanced user experience for both employer and jobseeker. It provides user friendly interface which facilitates in reaching wide range of audience. The application has achieved all the requirements that were initially set in the requirements gathering phase.

CHAPTER 8

FUTURE SCOPES

This project is also developed for the future uses.Adding more security and some extra modules this system can be used in every platform.This project full fills the primary requirements of the job seekers and employers. It can be extended in several ways – We can provide suggestions and email updates for new job postings based on the job seekers search history. Since, the job seekers might be interested in building a strong Resume, we can provide tips and information for the same. We can also provide templates for building the Resumes which might interest most applicants. The mobile application is developed fulfilling the functionalities of job seeker, it can be extended to support functionalities of Employer as well.

CHAPTER 9

REFERENCES

1.http://www.wikipedia.com

2.https://msdn.microsoft.com

3.http://1000 projects.com

4.http://csharp corner.com

5.http://scribd.com

6.http://www.mayoclinic.org

7.https://citycentreclinic.com